WEEK 5 PROJECT

PAYMENT TRANSACTION MANAGEMENT SYSTEM

Database Design, Transaction Operations & Revenue Analysis

Subject	Database Management System (MySQL)
Project	Week 5 – Payment Transaction Management System
Database	ecommerce
Relationship	Orders → Payment (one-to-zero-or-one)

This report documents the Payment table, constraints, sample transactions, payment operations, and payment/revenue analysis queries.

1. Objective

To extend the Week 4 Order Management System by storing payment information for each order, tracking payment modes and statuses, and generating transaction and revenue reports.

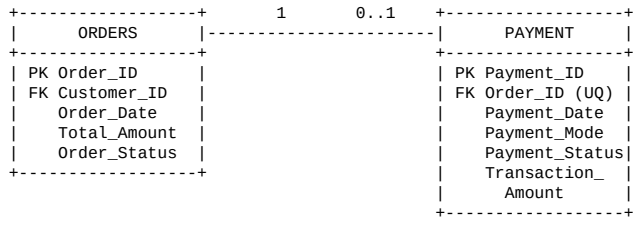
2. Payment Table Design

Column	Data Type	Constraint / Purpose
Payment_ID	INT	Primary key, auto-increment
Order_ID	INT	NOT NULL, UNIQUE, foreign key to Orders.Order_ID
Payment_Date	DATETIME	NOT NULL, defaults to current timestamp
Payment_Mode	VARCHAR(30)	NOT NULL, CHECK allowed payment modes
Payment_Status	VARCHAR(20)	NOT NULL, CHECK Pending/Success/Failed
Transaction_Amount	DECIMAL(10,2)	NOT NULL, CHECK amount >= 0

Constraints explained

• Primary Key: Payment_ID uniquely identifies a payment record.
• Foreign Key: Order_ID references Orders(Order_ID).
• UNIQUE: Order_ID is unique in Payment, so each order can have at most one payment record.
• NOT NULL: Required payment fields must have values.
• CHECK: Restricts payment modes, payment statuses, and non-negative transaction amounts.

3. Updated ER Diagram



Customer 1 ---- M Orders
Each Order has zero or one Payment record.

The UNIQUE constraint on Payment.Order_ID enforces at most one payment record per order. It does not require every order to have a payment record.

4. Sample Data

Sample customers and orders are taken from the Week 4 ecommerce database. Execute the Week 4 SQL script first, or use equivalent existing Customer and Orders tables.

Payment_ID	Order_ID	Payment Date	Mode	Status	Amount (■)
1	1001	2026-10-01	UPI	Success	46,300.00
2	1002	2026-10-03	Credit Card	Success	3,000.00
3	1003	2026-10-10	Debit Card	Failed	2,300.00
4	1004	2026-10-12	Net Banking	Success	45,000.00
5	1005	2026-10-15	Cash on Delivery	Pending	1,750.00

5. Payment Transaction Operations

The full SQL script is provided separately. Important queries are shown below; the script contains the complete table creation, sample inserts, CRUD examples, and all requested reports.

Create the Payment table

```
CREATE TABLE Payment (
  Payment_ID INT PRIMARY KEY AUTO_INCREMENT,
  Order_ID INT NOT NULL UNIQUE,
  Payment_Date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  Payment_Mode VARCHAR(30) NOT NULL
    CHECK (Payment_Mode IN ('UPI','Credit Card','Debit Card',
                            'Net Banking','Cash on Delivery')),
  Payment_Status VARCHAR(20) NOT NULL DEFAULT 'Pending'
    CHECK (Payment_Status IN ('Pending','Success','Failed')),
  Transaction_Amount DECIMAL(10,2) NOT NULL CHECK (Transaction_Amount >= 0),
  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
    ON UPDATE CASCADE ON DELETE CASCADE
);
```

Display successful payments and failed/pending transactions

```
SELECT * FROM Payment WHERE Payment_Status = 'Success';

SELECT * FROM Payment
WHERE Payment_Status IN ('Failed', 'Pending')
ORDER BY Payment_Date;
```

Update status after retry and retrieve one order's payment

```
UPDATE Payment
SET Payment_Status = 'Success', Payment_Date = CURRENT_TIMESTAMP
WHERE Order_ID = 1003 AND Payment_Status = 'Failed';

SELECT * FROM Payment WHERE Order_ID = 1001;
```

Display payment with customer and order details

```
SELECT p.Payment_ID, o.Order_ID, c.Customer_Name, o.Order_Date,
       o.Order_Status, p.Payment_Date, p.Payment_Mode,
       p.Payment_Status, p.Transaction_Amount
FROM Payment p
JOIN Orders o ON o.Order_ID = p.Order_ID
JOIN Customer c ON c.Customer_ID = o.Customer_ID
ORDER BY p.Payment_ID;
```

6. Payment Analysis & Revenue Reports

```
-- Number of transactions by payment mode
SELECT Payment_Mode, COUNT(*) AS Number_Of_Transactions
FROM Payment GROUP BY Payment_Mode
ORDER BY Number_Of_Transactions DESC;

-- Most preferred payment method (includes ties)
SELECT Payment_Mode, COUNT(*) AS Transaction_Count
FROM Payment GROUP BY Payment_Mode
HAVING COUNT(*) = (
  SELECT MAX(mode_count) FROM (
    SELECT COUNT(*) AS mode_count FROM Payment GROUP BY Payment_Mode
  ) AS mode_counts
);

-- Total successful payment revenue
SELECT COALESCE(SUM(Transaction_Amount),0) AS Successful_Revenue
FROM Payment WHERE Payment_Status = 'Success';

-- Revenue by payment method (successful only)
SELECT Payment_Mode, SUM(Transaction_Amount) AS Revenue
FROM Payment WHERE Payment_Status = 'Success'
GROUP BY Payment_Mode ORDER BY Revenue DESC;

-- Average transaction amount
SELECT ROUND(AVG(Transaction_Amount),2) AS Average_Transaction_Amount
FROM Payment;

-- Total successful, failed, and pending transactions
SELECT Payment_Status, COUNT(*) AS Transaction_Count,
       SUM(Transaction_Amount) AS Amount_Total
FROM Payment GROUP BY Payment_Status;

-- Orders with failed or pending payments
SELECT o.Order_ID, c.Customer_Name, p.Payment_Mode,
       p.Payment_Status, p.Transaction_Amount
FROM Orders o JOIN Customer c ON c.Customer_ID=o.Customer_ID
JOIN Payment p ON p.Order_ID=o.Order_ID
WHERE p.Payment_Status IN ('Failed','Pending');
```

7. Illustrative Payment Analysis Outputs

The following figures are calculated from the five sample payment records before running the optional retry/update examples. These are illustrative report results, not screenshots from a live MySQL Workbench session.

Metric	Sample result
Total payment records	5
Successful transactions	3
Failed transactions	1
Pending transactions	1
Successful payment revenue	■94,300.00
Average transaction amount (all statuses)	■19,670.00
Most preferred payment mode	Tie: each mode used once

Revenue by payment method (successful payments only)

Payment Mode	Successful Revenue (■)
UPI	46,300.00
Credit Card	3,000.00
Net Banking	45,000.00
Debit Card	0.00 (failed transaction excluded)
Cash on Delivery	0.00 (pending transaction excluded)

8. Query Output Screenshots

To add genuine output screenshots, open MySQL Workbench, execute the Week 4 database script first, then execute Week5_Payment_Transaction_Management.sql in sections. Run each report query and capture the result grid. Paste the screenshots into your final documentation if required.

9. How to Run and Submit

1. Run the Week 4 Order Management System script first so the ecommerce database, Customer table, and Orders table exist.
2. Open Week5_Payment_Transaction_Management.sql in MySQL Workbench and run the CREATE TABLE and sample INSERT sections.
3. Execute report queries individually. Do not re-run sample INSERT statements after they have already succeeded, because Payment_ID and Order_ID are unique.
4. Capture actual query output screenshots and add your name/college details if required.
5. Upload the PDF and SQL script to GitHub, then submit your repository link using the form provided by your instructor.

Important: The SQL script intentionally assumes the Week 4 ecommerce database. MySQL 8.0+ is recommended for CHECK constraints. The UNIQUE Order_ID rule prevents multiple payment records for the same order, as required.